
Linux Kernel Auditing Lab Material

Course: *Introduction to Vulnerability Research Lab*

Author name: *Dr. Bernard Lampe*

Due date: *November 7th, 2019*

Introduction

1. This lab pairs the **Linux Kernel Auditing** lecture with a runnable user-space model of a vendor driver's `ioctl/mmap` surface: `model\_driver.c`. The kernel reasoning (bounds checks, `copy_from_user` sizing, reference discipline) applies unchanged; only the primitives differ.

2. Build (verified on the reference host):

```
1 clang -g -O1 -fsanitize=address model_driver.c -o model_driver
```

Exercise 1 — Audit the Model Driver

1. Read `model\_driver.c` as if its `bank\_ioctl/bank\_mmap` were a driver's fops. Inventory: shared state, checks applied, ordering of validation vs. use.

2. Identify the two planted bugs *before* running anything:

- (a) a bounds check computed in the wrong width (u32 sum wraps: `0xFFFFFFFF0 + 0x40 == 0x30`)
- (b) a `copy\_from\_user` one byte short of the destination structure (uninitialized trailing field used as an access flag).

3. For each bug: which lecture bug-pattern class is it (integer/arithmetic vs. uninitialized)? What user-space preconditions does an attacker need?

Exercise 2 — Confirm Behaviorally

1. Run the built driver. Verified reference behavior:

```
1 ok call: 0
2 ==...==ERROR: AddressSanitizer: BUS on unknown address ...
3 ==...==The signal is caused by a WRITE memory access.
4     #1 ... in bank_mmap    model_driver.c:42
5     #2 ... in bank_ioctl  model_driver.c:56
6     #3 ... in main        model_driver.c:71
```

Exit status 134.

2. Explain in kernel terms what this crash corresponds to: a user-supplied descriptor whose wraparound check passed, driving a `memset` (write path) far past the device bank. In a real driver this is a write on kernel or device memory; in our model it is caught by the C library page fault and ASan.

3. Fix the check (use the widest type, or `size > len - offset` after proving `offset <= len`) and re-run until exit 0 with both calls clean.
4. Fix the short `copy\_from\_user` and reason: why does an uninitialized `rw` give an attacker a read-vs-write selector on kernel memory? (Uninitialized-user-blob bugs are privilege-primitive bugs, not just infoleaks.)

Exercise 3 — Real Kernel Triage Drills (paper)

1. Pick any two CVEs from the `android_questions.txt` surface list (`binder`, `ashmem`, `ION`, `GPU`) and for each: the component, the fops/ioctl verb reached, the bug class per the lecture, the primitive granted, and the elevation mechanism (`cred overwrite`, `modprobe\_path`, or shared-memory confusion).
2. Skill: connect the user-space crash hypothesis to a kernel primitive. Write both as the report pair (user-space symptom, kernel root cause) as described in the source auditing lecture's **From Bug to Report**.

Deliverables

1. Driver audit inventory (Exercise 1) with the two bugs labeled.
2. Fixed driver source, plus a one-line ASan session proving each fix (Exercise 2).
3. Two paper CVE triage reports (Exercise 3).